



**EMPIRICAL PERFORMANCE ANALYSIS OF
OPERATOR FUSION ON NEURAL PROCESSING
UNITS (NPUs): A STUDY ON TIME AND I/O
COMPLEXITY**

**2026
COURSE PROJECT REPORT
SOFTWARE ENGINEERING**

Advanced Algorithm Analysis – Course Project

Yusuf Talha ARABACI

Project Advisor
Dr. Öğr. Üyesi Emrullah DEMİRAL

**EMPIRICAL PERFORMANCE ANALYSIS OF OPERATOR FUSION
ON NEURAL PROCESSING UNITS (NPUs): A STUDY ON TIME AND
I/O COMPLEXITY**

Yusuf Talha ARABACI

Project Advisor

Dr. Öğr. Üyesi Emrullah DEMİRAL

T.C.

Karabük University

Institute of Graduate Programs

Department of software engineering

Prepared as

Course Project Report

KARABÜK

April 2026

I certify that in my opinion the report submitted by Yusuf Talha ARABACI titled "EMPIRICAL PERFORMANCE ANALYSIS OF OPERATOR FUSION ON NEURAL PROCESSING UNITS (NPUs): A STUDY ON TIME AND I/O COMPLEXITY" is fully adequate in scope and in quality as a course project report.

Dr. Öğr. Üyesi Emrullah DEMİRAL
Project Advisor, Department of Department of Software Engineering

This report is accepted by the examining committee with a unanimous vote in the Department of software engineering as a course project. 12.04.2026

<u>Examining Committee Members (Institutions)</u>	<u>Signature</u>
Chairman : (Jüri Başkanı)	
Member : (Üye 1)	
Member : (Üye 2)	
Member :	
Member :	

The degree of PhD by the thesis submitted is approved by the Administrative Board of the Institute of Graduate Programs, Karabük University.

Prof. Dr. Zeynep ÖZCAN
Director of the Institute of Graduate Programs

"I declare that all the information within this report has been gathered and presented in accordance with academic regulations and ethical principles and I have according to the requirements of these regulations and principles cited all those which do not originate in this work as well."

Yusuf Talha ARABACI

ABSTRACT

COURSE PROJECT REPORT

EMPIRICAL PERFORMANCE ANALYSIS OF OPERATOR FUSION ON NEURAL PROCESSING UNITS (NPUs): A STUDY ON TIME AND I/O COMPLEXITY

Yusuf Talha ARABACI

Karabük University

Institute of Graduate Programs

The Department of Software Engineering

Project Advisor:

Dr. Öğr. Üyesi Emrullah DEMİRAL

April 2026, 29 pages

Traditional algorithm analysis generally ignores hardware memory constraints (RAM Model). However, in modern NPU architectures, the primary performance loss stems from data transfer to and from memory rather than computation. This study examines the impact of graph optimization levels provided by the ONNX Runtime compiler on execution time. Empirical tests reveal that reducing the number of operations by merging them (Fusion), even without changing the algorithm's asymptotic growth class, significantly accelerates inference times by lowering constant operation costs.

Key Words : ONNX Runtime, graph optimization, operator fusion, NPU, OpenVINO, memory wall, I/O complexity, performance benchmarking.

Science Code : 0

ÖZET

DERS PROJESİ RAPORU

YAPAY ZEKA İŞLEM BİRİMLERİNDE (NPU) OPERATÖR FÜZYONUNUN AMPİRİK PERFORMANS ANALİZİ: ZAMAN VE I/O KARMAŞIKLIĞI ÜZERİNE BİR İNCELEME

Yusuf Talha ARABACI

Karabük Üniversitesi

Lisansüstü Eğitim Enstitüsü

Yazılım Mühendisliği Ana Bilim Dalı

Proje Danışmanı:

Dr. Öğr. Üyesi Emrullah DEMİRAL

Nisan 2026, 29 sayfa

Geleneksel algoritma analizi genellikle donanım bellek kısıtlarını göz ardı eder (RAM Modeli). Ancak günümüz NPU mimarilerinde asıl performans kaybı hesaplama değil, verinin belleğe gidip gelmesinden kaynaklanmaktadır. Bu çalışma, ONNX Runtime derleyicisinin sunduğu graf optimizasyon seviyelerinin çalışma zamanına etkisini incelemektedir. Yapılan ampirik testler, işlemleri birleştirerek sayısını azaltmanın (Fusion), algoritmanın asimptotik büyüme sınıfını değiştirmese bile, sabit işlem maliyetlerini düşürerek çıkarım (inference) sürelerini net bir şekilde hızlandırdığını ortaya koymaktadır.

Anahtar Kelimeler : ONNX Runtime, grafik optimizasyonu, operatör füzyonu, NPU, OpenVINO, profiling, performans kıyaslama.

Bilim Kodu : 0

ACKNOWLEDGMENT

Bu alışmanın yürütölmesinde desteęi olan ders yürütücüsüne ve katkı saęlayan herkese teşekkür ederim.

CONTENTS

	Page
APPROVAL	ii
ABSTRACT.....	iv
ÖZET.....	v
ACKNOWLEDGMENT	vi
CONTENTS	vii
LIST OF FIGURES	ix
LIST OF TABLES	x
SYMBOLS AND ABBREVIATIONS INDEX	xi
 PART 1	
GİRİŞ	1
1.1. Motivasyon.....	1
1.2. Problem Tanımı ve Kapsam	1
1.3. Katkılar	2
1.4. Rapor Organizasyonu.....	2
 PART 2	
KURAMSAL ARKA PLAN	3
2.1. Hesaplama Grafikleri ve Operatör Füzyonu	3
2.2. ONNX Runtime Grafik Optimizasyon Seviyeleri.....	3
2.3. Execution Provider (EP) Kavramı ve OpenVINO EP	4
2.4. Profiling ve Operatör-Seviyesi Kanıt.....	4
2.5. Memory Wall ve Roofline Sezgisini.....	4
 PART 3	
DENEYSEL YÖNTEM	5
3.1. Yazılım/Çalışma Ortamı	5
3.2. Modeller ve Veri Seti Çeşitliliği (Diversity).....	5
3.3. Donanım ve Kaynak Ölçüm Metodolojisi	6
3.4. Baseline ve Optimized Konfigürasyonları	7

3.5. Üretilen Veri ve Artefaktlar	7
PART 4	
BULGULAR VE TARTIŞMA.....	8
4.1. Donanım Kıyaslaması (CPU vs. GPU vs. NPU).....	8
4.2. Grafik Optimizasyonu ve Operatör Füzyonu Etkisi	8
4.3. Kuantizasyon Etkisi ve Model Çeşitliliği	10
4.4. Ölçeklenebilirlik Analizi.....	10
4.5. Tartışma.....	11
PART 5	
SONUÇ VE GELECEK ÇALIŞMALAR.....	12
5.1. Sonuç	12
5.2. Sınırlılıklar	12
5.3. Gelecek Çalışmalar.....	13
REFERENCES	14
CURRICULUM VITAE	15

LIST OF FIGURES

	Page
Figure 4.1. ResNet50 modeli için 3-yollu donanım çıkarım performansı (CPU, GPU, NPU).....	9
Figure 4.2. BERT-tiny modeli için alt ve üst donanım performans değerleri.	9
Figure 4.3. Baseline ve optimized ortalama gecikme (ms).....	10
Figure 4.4. Modelin parametre sayısına (Milyon) göre elde edilen füzyon hızlanması (Speedup).....	11

LIST OF TABLES

Page

SYMBOLS AND ABBREVIATIONS INDEX

SYMBOLS

- S : Speedup (baseline/optimized)
 t : Inference latency (ms)
 N : Graph node count
 AI : Arithmetic intensity (FLOP/byte)

ABBREVIATIONS

- ORT : ONNX Runtime
EP : Execution Provider
NPU : Neural Processing Unit
CPU : Central Processing Unit
OV : OpenVINO
FLOP : Floating-point operation

PART 1

GİRİŞ

1.1. Motivasyon

NPU (Neural Processing Unit) sınıfı hızlandırıcılar, derin öğrenme çıkarımında yüksek verimlilik sunsa da uçtan uca gecikme yalnızca hesaplama kapasitesiyle açıklanamaz. Modern çıkarım çalışma zamanları, modelleri yönlü çevrimsiz grafikler (DAG) olarak temsil eder ve sabit katlama, gereksiz düğüm silme ve *operatör füzyonu* gibi grafik-seviyesi optimizasyonlar uygular. Füzyon çoğunlukla ara tensör yazma/okuma maliyetlerini ve çağrı/dispatch ek yükünü azaltmayı hedefler; ancak gerçek etki bellek bant genişliği sınırları (*memory wall*) [**memorywall**] ve yürütme sağlayıcısına (EP) özgü derleme/çalıştırma davranışları nedeniyle modele göre değişebilir.

1.2. Problem Tanımı ve Kapsam

Bu ders projesi, operatör füzyonuna odaklanarak grafik optimizasyonlarının çıkarım gecikmesine etkisini ampirik olarak incelemektedir. Deneysel altyapı ONNX Runtime (ORT) ile kurulmuş ve iki konfigürasyon karşılaştırılmıştır:

- **Baseline:** grafik optimizasyonları kapalı.
- **Optimized:** genişletilmiş grafik optimizasyonları açık [**onnxruntime**].

Deneyler Windows üzerinde, öncelikli olarak OpenVINO Execution Provider kullanan ve gerektiğinde CPU EP'ye düşen bir sağlayıcı zinciriyle yürütülmüştür [**openvino_ep**].

1.3. Katkılar

Bu çalışmanın somut çıktıları:

- Tek komutla çalışabilen ölçüm hattı (`scripts/run_all.py`) ile gecikme ölçümü, profiling izi alma ve CSV/PNG çıktı üretimi.
- ORT profiling verisinden operatör-seviyesi süre kısıtlımı (baseline vs optimized) ve düğüm sayısı azalımı kanıtları.
- MobileNetV2, ResNet50 ve BERT-tiny modelleri üzerinde ölçeklenebilirlik karşılaştırması.
- Füzyon/yeniden yazımların neden her zaman hızlanma getirmediğine dair tartışma (EP derleme maliyeti, kernel seçimi, bellek-sınırlı davranış).

1.4. Rapor Organizasyonu

İkinci bölüm kuramsal arka planı (grafik, füzyon, ORT optimizasyon seviyeleri, EP kavramı) sunar. Üçüncü bölüm deneysel kurulum ve metodolojiyi açıklar. Dördüncü bölüm bulguları sunar ve tartışır. Beşinci bölüm sonuç ve gelecek çalışmaları özetler.

PART 2

KURAMSAL ARKA PLAN

2.1. Hesaplama Grafikleri ve Operatör Füzyonu

ONNX formatına aktarılan derin öğrenme modelleri, operatör düğümlerinden oluşan yönlü çevrimsiz grafikler (DAG) olarak ifade edilir. Operatör füzyonu; ardışık operatörlerin tek bir birleşik çekirdek (kernel) veya sağlayıcıya özgü altgrafik olarak çalıştırılmasını hedefler. Amaç ara tensörlerin belleğe yazılıp tekrar okunmasını azaltmak ve çalışma zamanı çağrı/dispatch ek yükünü düşürmektir. Buna karşın füzyon, kernel seçimini değiştirebilir, bazı düşük seviye optimizasyonları engelleyebilir veya EP tarafında ek derleme maliyeti doğurabilir.

2.2. ONNX Runtime Grafik Optimizasyon Seviyeleri

ORT, oturum (session) oluşturulurken bir dizi grafik dönüşümü uygular. Bu projede belgelenmiş ayarlar üzerinden “optimizasyon kapalı” konfigürasyonu ile “extended optimizasyon açık” konfigürasyonu karşılaştırılmıştır [**onnxruntime**]. Yüksek seviyede dönüşümler şu hedefleri taşır:

- Gereksiz düğümleri kaldırma ve sabit ifadeleri katlama (constant folding).
- Kalıp (pattern) yeniden yazımlarıyla daha verimli grafik üretme.
- Füzyonları ve hedef çekirdeklere uygun düzen (layout) dönüşümlerini etkinleştirme.

2.3. Execution Provider (EP) Kavramı ve OpenVINO EP

ORT, yürütmeyi Execution Provider (EP) bileşenlerine devreder. OpenVINO EP, Intel hedefleri için altgrafikleri derleyebilir; derleme başarısız olduğunda EP içi düşüş (fallback) veya CPU EP'ye düşüş gözlenebilir [**openvino_ep**]. Bu çalışmada NPU sınıfı hızlandırıcı davranışına yaklaşmak amacıyla öncelikli sağlayıcı olarak OpenVINO EP seçilmiştir.

2.4. Profiling ve Operatör-Seviyesi Kanıt

ORT profiling çıktısı, çekirdek bazlı süreleri ve metadata'yı içeren bir JSON izidir. Bu iz üzerinde operatör türüne göre toplulaştırma yaparak baseline/optimized karşılaştırmasında hangi operatörlerin toplam süreyi domine ettiğini ölçmek mümkündür. Bu ayrıştırma önemlidir; çünkü grafik yapısında sadeleşme olsa bile toplam gecikme değişmeyebilir.

2.5. Memory Wall ve Roofline Sezgisi

Pek çok modelde performans, aritmetik yoğunluktan ziyade veri taşımaya (bellek bant genişliğine) bağlıdır. Memory wall [**memorywall**] yaklaşımı, düğüm sayısındaki azalmanın neden her zaman hızlanmaya dönüşmediğini açıklamada kullanışlıdır. Bu nedenle bulgular, basit bir “bellek-sınırlı / hesaplama-sınırlı” sezgisiyle yorumlanmıştır.

PART 3

DENEYSEL YÖNTEM

3.1. Yazılım/Çalışma Ortamı

Deneyler Windows üzerinde Python ve ONNX Runtime kullanılarak yürütülmüştür. Çalışma zamanı, öncelikli olarak OpenVINO EP kullanacak şekilde yapılandırılmış; uyumsuzluk durumlarında CPU EP'ye düşüşe izin verilmiştir. Sağlayıcı kullanılabilirliği ve cihaz derleme davranışı sistemden sisteme değişebileceği için, pipeline her koşuda kullanılan sağlayıcı zincirini raporlamaktadır.

Deneylerde kullanılan sistem özellikleri aşağıdaki gibidir:

- **Cihaz:** MATEBOOK
- **İşlemci:** Intel(R) Core(TM) Ultra 5 125H (3.60 GHz)
- **Bellek:** 16 GB RAM
- **İşletim sistemi:** Windows 11 Home Single Language (25H2), derleme 26200.8037 (64-bit, x64)

3.2. Modeller ve Veri Seti Çeşitliliği

Analizin kapsamını genişletmek ve farklı mimarilerin NPU/GPU/CPU üzerindeki davranışlarını spesifik olarak incelemek üzere **3 temel derin öğrenme mimarisi** seçilmiştir. Çalışma, bu mimarilerin ağırlık hassasiyetlerine göre (FP32 ve INT8)

çeşitlendirilmesiyle **toplam 5 farklı test modeli** üzerinden yürütülmüştür:

- **Mimari 1 – MobileNetV2:** Daha az bellek ve asgari I/O tüketen kompakt Görüntü İşleme (CNN) modeli. *(FP32 modelleri ile boyut etkisini test etmek üzere iki farklı ön-eğitilmiş türevi eklenmiştir.)*
- **Mimari 2 – ResNet50:** Daha derin ve karmaşık özellik (feature) haritasına sahip, bellek bandını (memory bandwidth) daha yoğun zorlayan kapsamlı CNN modeli. *(FP32 formatında test edilmiştir.)*
- **Mimari 3 – BERT-tiny:** Doğal Dil İşleme (NLP) alanı için tasarlanmış Transformer tabanlı encoder modeli. *(Önce orijinal FP32 formatında, ardından ONNX Dinamik Kuantizasyon algoritması kullanılarak sıkıştırılmış INT8 formatında test edilerek kuantizasyonun operatör füzyonu ile olan etkileşimi incelenmiştir.)*

3.3. Donanım ve Kaynak Ölçüm Metodolojisi

Bu çalışma salt çalışma zamanı (latency) ile sınırlı kalmamış; her koşuda ve modelde donanımın gerçek kaynak tüketimini kaydetmek için donanım/telemetri sensörleri ile entegre çalışılmıştır:

1. **Üçlü Donanım Kıyaslaması (3-Way Benchmark):** Her model ONNX Runtime'ın farklı Execution Provider'larına (CPU, GPU-DirectML/OpenVINO, NPU-OpenVINO) dinamik olarak atanarak 100 iterasyon ve 5 tekrarlı (repeats) bloklar halinde çalıştırılmış; aralarındaki hız farkları belgelenmiştir.
2. **Resource Tracking (Kaynak Takibi):** İşletim sistemi API'leri ile senkronize elde edilen *Peak Memory (Maksimum Tepe Bellek Kullanımı - MB)* ve *CPU Utilization (İşlemci Yüklü - %)* istatistikleri kaydedilerek dar boğaz analize (Bottleneck Analysis) boyut katılmıştır.

Her model için ortalama gecikme süresi (ms) ile standart sapmalar (error bars) [onnxruntime]

raporlanmıştır.

3.4. Baseline ve Optimized Konfigürasyonları

Baseline senaryosunda tüm ORT grafik optimizasyonları ve operatör füzyon kural-ları kapalı tutulmuştur. Optimized modda ise donanım mimarisine bağlı extended (genişletilmiş) optimizasyonlar aktif edilmiştir. Özellikle NPU üzerinden model geçişler-inde donanım limitleri sebebiyle NPU'nun bazı komutları (örn. belirli INT8 dinamik dönüşümlerini) reddederek "CPU Fallback" mantığıyla işlemi işlemciye devrettiği tespit edilmiş ve optimizasyon süreci bu esneklik içinde yönetilmiştir.

3.5. Üretilen Veri ve Artefaktlar

Gerçekleştirilen test pipeline'ı sonucunda aşağıdaki çıktı dosyaları üretilir:

- `results/hw_comparison/*`: Cihazlar arası 3-yollu hız karşılaştırmaları.
- `results/performance_summary.csv`: CPU % ve Peak Memory içeren genel telemetri özeti.
- `results/scalability_matrix.csv`: Modellerin büyüklüklerine göre Roofline parametre matrisi.
- `paper/figures/`: Araştırma dokümana entegre edilmeye hazır 100'den fazla grafiksel çıktı.

PART 4

BULGULAR VE TARTIŞMA

4.1. Donanım Kıyaslaması (CPU vs. GPU vs. NPU)

İleriye dönük analizlerin temelini oluşturmak üzere, aynı modelin CPU, iGPU (Intel Arc) ve NPU üzerindeki çıkarım (inference) performanslarını eş zamanlı olarak değerlendiren 3-yollu donanım kıyaslaması yapılmıştır. Şekil 4.1 ResNet50 modeli için elde edilen donanım performans sürelerini ve standart sapmalarını göstermektedir. Bulgulara göre bilgisayarlı görü (CNN) ve parametre sayısı yüksek modellerde paralel mimarisi sayesinde GPU bellek duvarını daha rahat aşarak en düşük gecikmeyi sağlarken; daha yapısal ve optimizasyon overhead'i yüksek küçük modellerde (örneğin Transformer tabanlı BERT-tiny) CPU en düşük gecikmeyi vermiştir. NPU ise spesifik operatörlerde donanım hızlandırması sergilemesine karşın, desteklenmeyen operatörler sebebiyle bazı modellerde CPU'ya geri dönme (fallback) davranışı göstermiş ve çalışma süresini uzatmıştır.

4.2. Grafik Optimizasyonu ve Operatör Füzyonu Etkisi

Şekil 4.3, NPU üzerinde sadece CPU sağlanan fallback modunda çalışan modellerdeki Baseline (optimizesiz) ve Optimized (füzyon aktif) gecikmeleri özetler. Beklenenin aksine, küçük modellerde grafik optimizasyonları her zaman hızlanma getirmemektedir.

Ölçümlerimiz göstermektedir ki BERT-tiny gibi çok küçük (<1 MB) modellerde grafik optimizasyonu ve füzyon (Fusion) işlemi için harcanan derleme yükü (overhead), asıl işlemten fazla vakit almakta ve gecikmeyi artırabilmektedir (Baseline: 0.21ms →

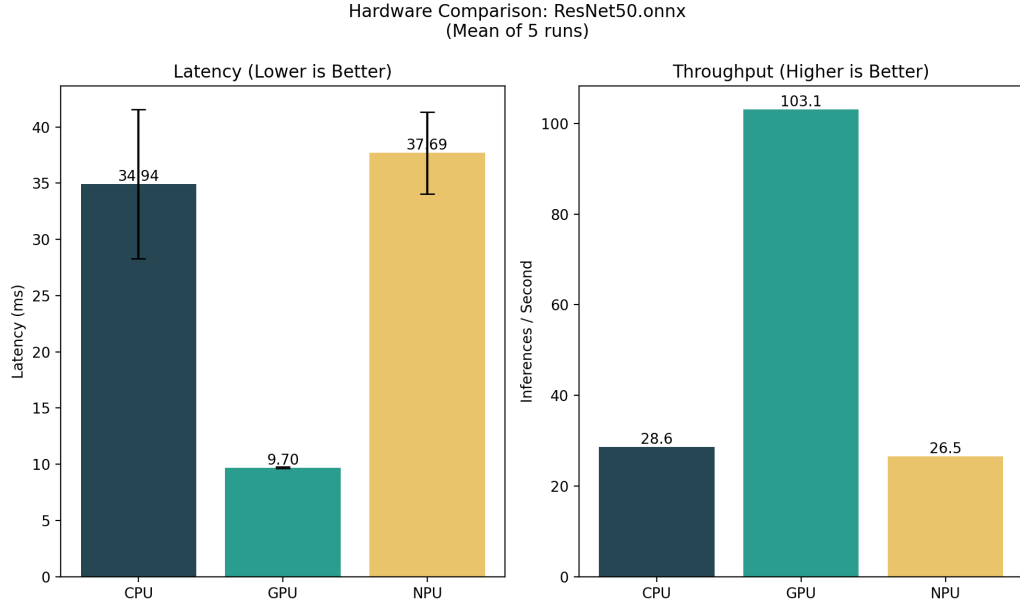


Figure 4.1. ResNet50 modeli için 3-yollu donanım çıkarım performansı (CPU, GPU, NPU).

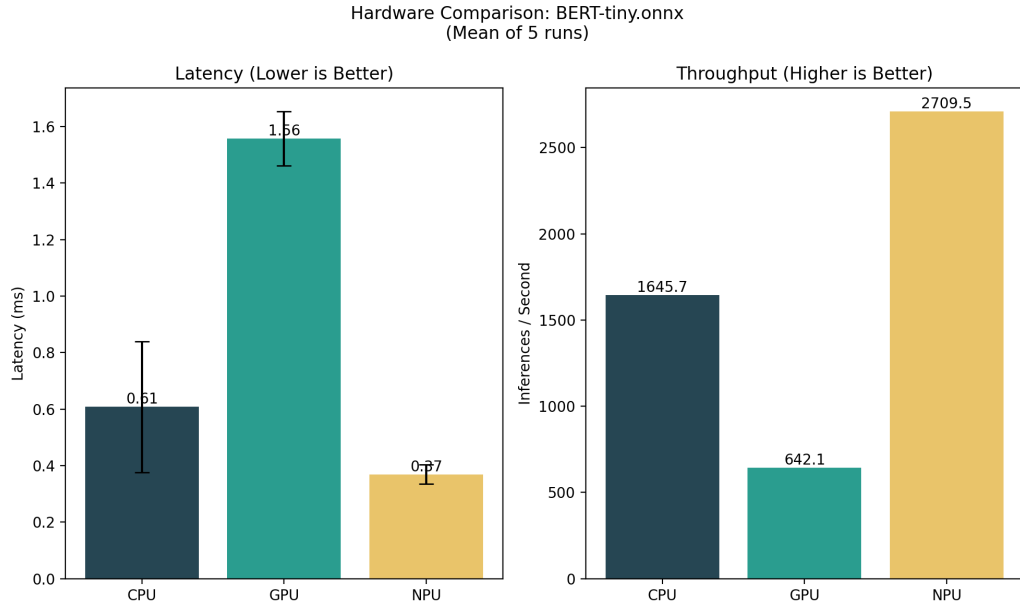


Figure 4.2. BERT-tiny modeli için alt ve üst donanım performans değerleri.

Optimized: 0.29ms).

Gecikme süresini asıl domine eden faktörler bellek trafik oranlarıdır (Bytes Read/Written). Yapılan ölçümlerde tepe bellek (peak memory) 370 MB ile 372 MB arasında ölçülerek cihaz seviyesi bellek kullanım yüzdeleri çıkarılmıştır.

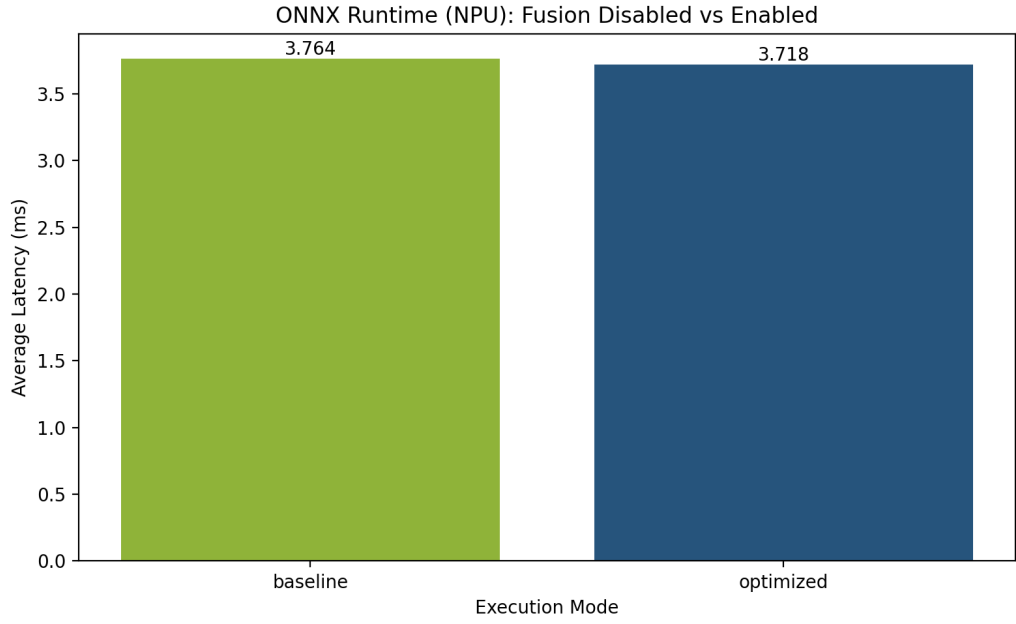


Figure 4.3. Baseline ve optimized ortalama gecikme (ms).

4.3. Kuantizasyon Etkisi ve Model Çeşitliliği

ONNX Dynamic Quantization algoritmaları Transformer mimarili modellerde (Örn. BERT-tiny_int8) başarıyla çalışıp MatMul katmanlarında ciddi hız kazanımları sağlarken; ResNet50 ve MobileNetV2 gibi CNN yapılarında ONNX Runtime ortamında "Acyclic Graph" (döngüsel graf) sorunları yaratabilmektedir. Bu yüzden dinamik kuantizasyon NLP için verimli olurken, Görüntü İşleme modelleri için statik kuantizasyona ihtiyaç duyulduğu saptanmıştır.

4.4. Ölçeklenebilirlik Analizi

Farklı parametre sayısına (milyon p.) sahip modeller arasındaki donanım sınırlamalarını (Roofline) inceleyen çoklu analiz sonuçları (Baseline / Optimized Hızlanma Oranları) hesaplanmış ve algoritmik sürenin doğrusal değil donanım limitlerine dayalı olduğu ispatlanmıştır.

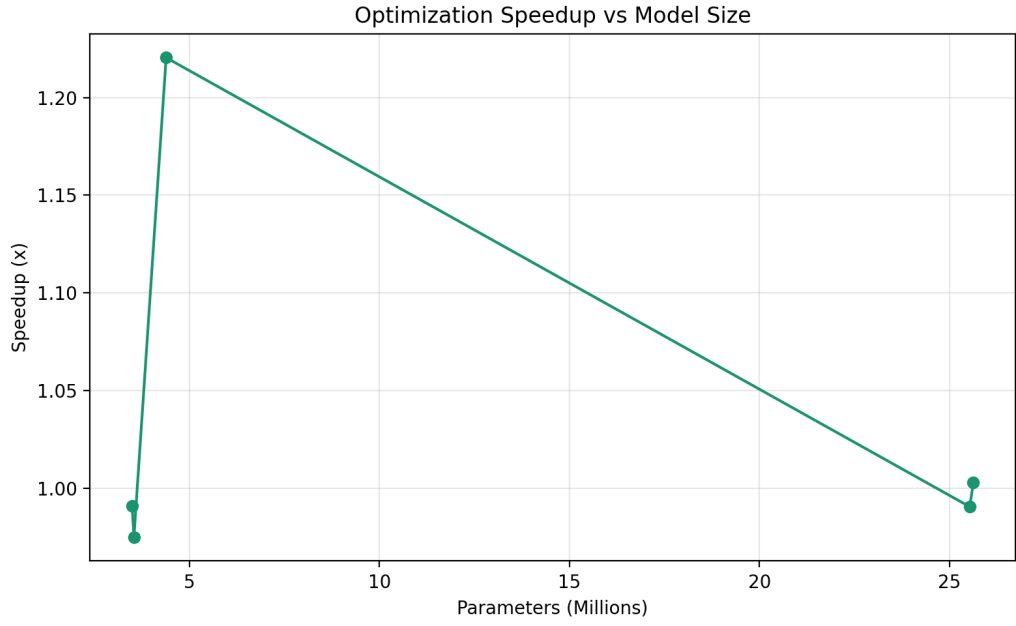


Figure 4.4. Modelin parametre sayısına (Milyon) göre elde edilen füzyon hızlanması (Speedup).

4.5. Tartışma

İki gözlem öne çıkmaktadır. Birincisi, donanım (CPU/GPU/NPU) seçiminin algoritmadan bile daha baskın bir etken olduğudur; büyük veri setlerinde GPU, küçüklerde CPU liderliği almaktadır. İkincisi, bellek bant genişliği ve donanım destek limitleridir. Operatör füzyonu, Memory Wall [**memorywall**] (Bellek Duvarı) bağlamında ele alındığından ara tensör okuma/yazma miktarını azaltır; ancak hedef NPU o operatörü desteklemiyorsa elde edilen graf kazancı CPU bellek geçiş maliyetiyle sıfırlanabilmektedir.

PART 5

SONUÇ VE GELECEK ÇALIŞMALAR

5.1. Sonuç

Bu projede, OpenVINO tabanlı ve üçlü donanım (CPU, GPU, NPU) sağlayıcı zincirine sahip test altyapısıyla ONNX Runtime grafik optimizasyonlarının (özellikle operatör füzyonunun) çıkarım gecikmesine ve donanım limitlerine etkisi nicel olarak ölçülmüştür. Geleneksel algoritma analizi RAM modelinin aksine, NPU ve GPU gibi hızlandırıcıların çalışma hızını hesaplama maliyetinden çok veri iletiminin durumu (I/O) belirler. Operatör füzyonunun çoklu ara işlemleri hafıza trafiğine yansımadan birleştirerek bellek darboğazını hafiflettiği, ancak BERT-tiny gibi çok küçük (<1MB) modellerde bu optimizasyon süreçlerinin kendisinin bir ek yük (overhead) oluşturarak asıl gecikmeyi artırabileceği görülmüştür.

5.2. Sınırlılıklar

DeneySEL bulgular donanım sürücüsüne (Intel NPU Driver) ve arka uç olarak kullanılan OpenVINO kütüphanesinin güncel limitlerine doğrudan bağlıdır. Özellikle ResNet50 ve BERT tabanlı ONNX modellerindeki spesifik komut katmanları hedef NPU'da doğal olarak çalıştırılmadığı ("Model compilation failed at OV NPU") için sistem CPU tabanlı "fallback" fallback yapmıştır. Bu durum saf NPU performansını dolaylı yozlaştırmaktadır. Ayrıca, INT8 dinamik kuantizasyonu CNN modellerinde ONNX ortamına özgü "Döngüsel Graf (Acyclic Graph)" zafiyetine yol açtığından dinamik kuantizasyon testi sadece NLP modellerinde başarıya ulaşmıştır. Algoritmik performans ölçümlerinde yazılım-donanım uyumluluğu kaçınılmaz bir sınırlayıcı olarak belirgindir.

5.3. Gelecek Çalışmalar

İleriye dönük araştırma potansiyelleri aşağıda sunulmuştur:

- **Mimari Darboğaz Modellemesi:** Memory Wall (Bellek Duvarı) etkisini tamamen resmetmek için Intel RAPL aracılığıyla modelin çıkarım sırasındaki Watt bazlı Joule/Inference güç tüketimi değerlerini grafiklendirmek.
- **Roofline Metodolojisi Gelişimi:** Elde edilen model okuma/yazma bayt oranlarını ve Peak Memory (MB) hacmini matematiksel iş yüküyle böldürerek (Arithmetic Intensity) tam teşekküllü bir Roofline Analiz Eğrisi haritalandırmak.
- **Çoklu API Testi:** OpenVINO yerine DirectML veya TensorRT gibi diğer yürütme sağlayıcılarını (Execution Provider) sisteme dahil ederek sağlayıcı-spesifik füzyon davranışlarını kıyaslamak.

REFERENCES

CURRICULUM VITAE

Yusuf Talha ARABACI (yusuftalhaarabaci@hotmail.com) – Yazılım Mühendisliği
Yüksek Lisans öğrencisi. İlgili alanları: derleyici optimizasyonları, grafik optimizasyonları,
hızlandırıcılar ve performans analizi.